

Software Requirements Specification (SRS)

Plant Care Scheduler System

Version: 1.0

Date: August 13, 2025

Document Type: Complete Application Specification

Table of Contents

- [1. Introduction](#)
 - [2. Overall Description](#)
 - [3. Specific Requirements](#)
 - [4. System Architecture](#)
 - [5. Security Requirements](#)
 - [6. Performance Requirements](#)
 - [7. UI/UX Requirements](#)
 - [8. Appendices](#)
-

1. Introduction

1.1 Purpose

The Plant Care Scheduler is a comprehensive web application designed to help plant enthusiasts, professional gardeners, and botanical organizations manage their plant care routines, track plant health, and optimize growing conditions. The system provides role-based access for plant owners, care specialists, and administrators, enabling collaborative plant management with comprehensive tracking, scheduling, and analytics capabilities.

1.2 Scope

The system enables users to manage their complete plant care journey while allowing specialists to provide expert guidance and administrators to oversee system operations. Key functionalities include:

- Plant Management:** Comprehensive plant profiles, species database, and care requirement tracking
- Care Scheduling:** Automated watering, fertilizing, pruning, and maintenance reminders
- Health Monitoring:** Growth tracking, disease detection, and treatment management
- Environmental Tracking:** Light, humidity, temperature, and soil condition monitoring
- Specialist Consultation:** Expert guidance, care plan development, and problem-solving support
- Community Features:** Plant sharing, experience exchange, and social gardening

- **Analytics Dashboard:** Growth patterns, care effectiveness, and optimization insights
- **Notification System:** Smart reminders, care alerts, and emergency notifications
- **Resource Library:** Plant care guides, troubleshooting resources, and best practices
- **Inventory Management:** Tools, supplies, and fertilizer tracking

1.3 Definitions, Acronyms, and Abbreviations

- **API:** Application Programming Interface
- **REST:** Representational State Transfer
- **JWT:** JSON Web Token
- **RBAC:** Role-Based Access Control
- **CRUD:** Create, Read, Update, Delete
- **JPQL:** Java Persistence Query Language
- **IoT:** Internet of Things
- **PPM:** Parts Per Million
- **pH:** Potential of Hydrogen
- **NPK:** Nitrogen, Phosphorus, Potassium
- **HVAC:** Heating, Ventilation, and Air Conditioning

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Spring Boot Security Documentation
- React.js Documentation
- Botanical Care Standards and Guidelines
- Smart Agriculture IoT Integration Standards

1.5 Overview

This document provides comprehensive requirements for the Plant Care Scheduler system, including functional specifications, security requirements, role-based access control, API definitions, and implementation guidelines for building a complete plant management platform.

2. Overall Description

2.1 Product Perspective

The Plant Care Scheduler is a standalone web application built using Spring Boot for backend services with Spring Security, React.js for frontend interface, and JWT-based authentication. The system

implements comprehensive role-based access control designed specifically for plant care management and horticultural operations.

2.2 Product Functions

- **Plant Profile Management:** Detailed plant information, species characteristics, and growth tracking
- **Automated Care Scheduling:** Smart reminders for watering, fertilizing, pruning, and repotting
- **Health Assessment Tools:** Disease identification, pest management, and recovery tracking
- **Environmental Monitoring:** Climate control, light optimization, and growing condition analysis
- **Care Plan Development:** Customized care routines, seasonal adjustments, and growth optimization
- **Specialist Collaboration:** Expert consultations, professional guidance, and advanced care strategies
- **Community Engagement:** Plant sharing, knowledge exchange, and social gardening features
- **Analytics and Insights:** Growth analysis, care effectiveness tracking, and optimization recommendations
- **Resource Management:** Tool tracking, supply inventory, and care material management
- **Notification Management:** Smart alerts, deadline reminders, and emergency notifications
- **Educational Content:** Care guides, best practices, and troubleshooting resources

2.3 User Classes and Characteristics

- **System Administrator:** Full system access, user management, system configuration, and platform oversight
- **Plant Care Specialist:** Expert guidance, care plan development, consultation services, and professional plant management
- **Premium Plant Owner:** Advanced features, specialist access, detailed analytics, and priority support
- **Standard Plant Owner:** Basic plant management, care scheduling, health tracking, and community access
- **Community Member:** Limited access to forums, shared content, and basic plant information
- **Guest User:** Public content access, basic plant information, and registration functionality

2.4 Operating Environment

- **Server:** Spring Boot 3.x with Spring Security (Java 17+)
- **Database:** JPA-compatible database with plant care management schema
- **Frontend:** React.js with responsive design (port 3000)
- **Backend:** Spring Boot with security (port 8080)
- **Authentication:** JWT-based authentication with role-based access control
- **IoT Integration:** Sensor data collection and environmental monitoring support

2.5 Design and Implementation Constraints

- **RESTful API Design:** Comprehensive API with security features and real-time capabilities
 - **JWT Authentication:** Required for all protected operations with role-based tokens
 - **Role-Based Access Control:** Mandatory implementation with hierarchical permissions
 - **Data Validation:** Comprehensive input validation for plant care data
 - **Real-time Monitoring:** Mandatory for environmental tracking and care alerts
 - **Mobile Responsive:** Optimized design for mobile plant care activities
 - **IoT Integration:** Support for sensor data and automated monitoring
 - **Botanical Standards:** Compliance with botanical naming conventions and care standards
-

3. Specific Requirements

3.1 External Interfaces

3.1.1 User Interfaces

- **Landing Page:** Plant care overview with user registration and login options
- **Plant Owner Dashboard:** Plant collection, care calendar, and health monitoring
- **Plant Profile Management:** Detailed plant information, care history, and photo gallery
- **Care Scheduler:** Calendar integration with automated reminders and task management
- **Health Monitor:** Disease detection, treatment tracking, and recovery documentation
- **Specialist Console:** Client management, consultation tools, and expert guidance features
- **Admin Dashboard:** User management, system analytics, and configuration tools
- **Community Hub:** Plant sharing, forums, and knowledge exchange platform
- **Resource Library:** Care guides, troubleshooting tools, and educational content

3.1.2 Hardware Interfaces

- **IoT Sensor Integration:** Soil moisture, light, temperature, and humidity sensors
- **Camera Integration:** Plant photography and growth documentation
- **Environmental Control:** Smart irrigation and lighting system integration

3.1.3 Software Interfaces

- **Secured REST API:** JWT authentication with comprehensive plant care endpoints
- **Weather Service Integration:** Climate data and outdoor condition monitoring
- **IoT Platform Integration:** Sensor data collection and automated monitoring
- **Notification Services:** Email, SMS, and push notification delivery

- **Calendar Integration:** Google Calendar, Outlook calendar synchronization

3.1.4 Communication Interfaces

- **HTTPS Protocols:** SSL/TLS encryption for all communications
- **JSON Data Format:** Standardized API communication
- **JWT Token Authentication:** Secure session management
- **WebSocket Support:** Real-time sensor data and live notifications

3.2 Functional Requirements

3.2.1 User Authentication and Account Management

FR1: Comprehensive User Registration and Authentication System

Description: The system shall provide secure user registration and authentication mechanisms that support multiple user roles, profile verification, and comprehensive account management for plant care activities.

Detailed Requirements:

- Registration form shall capture complete profile information including name, email, location, gardening experience level, and plant care interests
- Email verification shall be required with plant care welcome sequences
- User profile creation shall include gardening experience, plant preferences, growing conditions, and notification settings
- Multi-role support for plant owners, specialists, and administrators
- Password security with complexity requirements and periodic updates
- Account recovery with secure verification methods
- Profile completeness tracking with improvement recommendations

FR2: Role-Based Access Control for Plant Care Management

Description: The system shall implement sophisticated role-based access control tailored for plant care management, ensuring appropriate access levels while maintaining collaborative capabilities between users and specialists.

Detailed Requirements:

- **Standard Plant Owner:** Plant management, basic care scheduling, health tracking, community access
- **Premium Plant Owner:** Advanced analytics, specialist consultations, priority features, detailed insights

- **Plant Care Specialist:** Client management, expert guidance, consultation tools, professional plant management
- **System Administrator:** Complete system access, user management, configuration, analytics oversight
- **Community Member:** Forum access, shared content, basic plant information, social features
- **Guest User:** Limited plant information, registration access, and public content browsing

3.2.2 Plant Management System

FR3: Comprehensive Plant Profile and Collection Management

Description: The system shall provide sophisticated plant profile management that allows users to create detailed plant records, track growth, and maintain comprehensive plant collections.

Detailed Requirements:

- Plant profile creation with species identification, common name, scientific name, and care requirements
- Growth tracking with measurements, photos, and developmental milestones
- Environmental preference recording including light requirements, water needs, temperature ranges, and soil preferences
- Care history documentation with watering records, fertilization schedule, and treatment history
- Photo gallery with timestamped growth documentation and visual health tracking
- Plant categorization by location, care difficulty, and plant type
- Collection organization with grouping, tagging, and filtering capabilities
- Plant sharing features for community engagement and knowledge exchange

FR4: Species Database and Care Intelligence

Description: The system shall maintain a comprehensive species database with care requirements, common issues, and expert guidance for optimal plant health.

Detailed Requirements:

- Extensive species database with care requirements, growth patterns, and seasonal needs
- Care recommendation engine based on species, environment, and user experience
- Common problem identification with symptoms, causes, and treatment recommendations
- Seasonal care adjustment suggestions and environmental optimization tips
- Compatibility analysis for multi-plant environments and companion planting
- Regional adaptation guidance based on climate and growing conditions
- Expert care tips and professional recommendations for each species

3.2.3 Care Scheduling and Task Management

FR5: Automated Care Scheduling and Reminder System

Description: The system shall provide intelligent care scheduling that automates plant care routines, generates smart reminders, and adapts to plant needs and user preferences.

Detailed Requirements:

- Automated care schedule generation based on plant species, age, season, and environmental conditions
- Smart reminder system for watering, fertilizing, pruning, repotting, and pest inspection
- Task prioritization based on plant health, care urgency, and user availability
- Schedule adaptation based on environmental changes, plant response, and care feedback
- Bulk care management for large plant collections with efficient task grouping
- Care calendar integration with personal calendars and scheduling systems
- Completion tracking with care verification and outcome documentation
- Emergency care alerts for critical plant health situations

FR6: Health Monitoring and Assessment System

Description: The system shall provide comprehensive plant health monitoring capabilities that track plant condition, identify issues, and guide treatment decisions.

Detailed Requirements:

- Health assessment tools with visual inspection guides and symptom identification
- Disease and pest identification with photo comparison and expert guidance
- Treatment tracking with medication records, progress monitoring, and recovery documentation
- Growth analysis with measurement tracking, development milestones, and comparative analysis
- Environmental impact assessment with condition correlation and optimization recommendations
- Health trend analysis with historical tracking and predictive insights
- Recovery planning with treatment schedules and progress monitoring
- Specialist consultation integration for complex health issues

3.2.4 Environmental Monitoring and Optimization

FR7: Environmental Tracking and Smart Monitoring System

Description: The system shall provide comprehensive environmental monitoring that tracks growing conditions, integrates with smart sensors, and optimizes plant environments.

Detailed Requirements:

- IoT sensor integration for automated monitoring of soil moisture, light levels, temperature, and humidity
- Environmental data logging with historical tracking and trend analysis
- Automated alerts for suboptimal conditions and environmental emergencies
- Climate optimization recommendations based on plant requirements and current conditions
- Seasonal adaptation guidance with environmental adjustment suggestions
- Multi-zone monitoring for different growing areas and plant groupings
- Energy efficiency tracking for artificial lighting and climate control systems
- Weather integration with outdoor condition monitoring and forecast-based care adjustments

3.2.5 Specialist Consultation and Expert Services**FR8: Plant Care Specialist Collaboration System**

Description: The system shall provide comprehensive tools for plant care specialists to offer consultations, develop care plans, and provide expert guidance to plant owners.

Detailed Requirements:

- Specialist profile management with credentials, expertise areas, and service offerings
- Consultation scheduling with calendar integration and session management
- Client plant collection access with permission-based viewing and care plan development
- Expert guidance delivery through care plan creation, treatment recommendations, and ongoing support
- Communication tools with messaging, video consultation, and progress tracking
- Knowledge sharing platform with expert articles, care guides, and best practices
- Performance analytics with client success tracking and service effectiveness measurement
- Certification and credential verification for specialist authenticity

3.2.6 Community and Social Features**FR9: Plant Community and Knowledge Sharing Platform**

Description: The system shall provide community features that enable plant enthusiasts to share experiences, exchange knowledge, and build social connections around plant care.

Detailed Requirements:

- Community forums with topic-based discussions, Q&A sections, and expert participation
- Plant sharing marketplace with plant exchange, care material trading, and local connections

- Achievement system with care milestones, plant collection goals, and expertise recognition
- Knowledge base contribution with user-generated content, care tips, and experience sharing
- Local community features with regional groups, meetups, and garden tours
- Expert verification system for reliable advice and trusted community members
- Content moderation with quality control and community guidelines enforcement
- Social features with following, messaging, and collaboration tools

3.3 Advanced System Features

3.3.1 Analytics and Intelligence

FR10: Plant Care Analytics and Optimization System

Description: The system shall provide comprehensive analytics and intelligence capabilities that analyze care patterns, plant health, and optimization opportunities.

Detailed Requirements:

- **Care Effectiveness Analysis:** Success rate tracking, care pattern analysis, growth correlation studies
- **Plant Health Intelligence:** Health trend analysis, early warning systems, preventive care recommendations
- **Environmental Optimization:** Condition analysis, energy efficiency tracking, climate optimization insights
- **User Behavior Analytics:** Care consistency tracking, engagement patterns, learning progress analysis
- **Predictive Analytics:** Growth forecasting, care need prediction, health issue prevention
- **Comparative Analysis:** Plant performance comparison, care method effectiveness, seasonal adaptation
- **Resource Optimization:** Supply usage tracking, cost analysis, efficiency recommendations
- **Success Metrics:** Plant survival rates, growth achievements, care goal completion

3.3.2 Automation and Smart Features

FR11: Intelligent Automation and Smart Care System

Description: The system shall provide advanced automation capabilities that enhance care efficiency, reduce manual tasks, and optimize plant health outcomes.

Detailed Requirements:

- **Smart Care Scheduling:** AI-driven schedule optimization, adaptive timing, intelligent task prioritization

- **Automated Monitoring:** Sensor-based condition tracking, automated alerts, emergency response systems
- **Predictive Care:** Machine learning-based care prediction, proactive recommendations, preventive interventions
- **Smart Notifications:** Context-aware alerts, priority-based messaging, intelligent reminder timing
- **Care Automation:** Integration with smart irrigation, lighting control, climate management systems
- **Health Prediction:** Early disease detection, stress indicator analysis, intervention recommendations
- **Resource Management:** Automated supply tracking, reorder suggestions, inventory optimization
- **Learning Adaptation:** System learning from user behavior, care outcomes, and plant responses

3.4 Security and Data Protection

FR12: Comprehensive Security and Privacy Protection System

Description: The system shall implement enterprise-grade security measures ensuring data protection, privacy compliance, and secure operations for sensitive plant care information.

Detailed Requirements:

- **Data Encryption:** AES-256 encryption for sensitive plant and user data at rest and in transit
 - **Authentication Security:** Multi-factor authentication options, session management, device tracking
 - **Access Control:** Role-based permissions, resource-level security, audit logging
 - **Privacy Compliance:** GDPR compliance, data retention policies, consent management
 - **Security Monitoring:** Intrusion detection, vulnerability scanning, security incident response
 - **Data Backup:** Automated backups, disaster recovery, business continuity planning
 - **IoT Security:** Secure sensor communication, device authentication, data validation
-

4. System Architecture

4.1 Backend Architecture

4.1.1 Spring Boot Application Structure

```
src/main/java/com/plantcare/
├── PlantCareApplication.java
├── config/
│   ├── SecurityConfig.java
│   ├── JwtConfig.java
│   ├── WebSocketConfig.java
│   └── IoTConfig.java
├── controller/
│   ├── AuthController.java
│   ├── PlantController.java
│   ├── CareScheduleController.java
│   ├── HealthMonitorController.java
│   ├── EnvironmentController.java
│   ├── SpecialistController.java
│   ├── CommunityController.java
│   └── AnalyticsController.java
├── service/
│   ├── AuthService.java
│   ├── PlantService.java
│   ├── CareScheduleService.java
│   ├── HealthMonitorService.java
│   ├── EnvironmentService.java
│   ├── NotificationService.java
│   ├── SpecialistService.java
│   └── AnalyticsService.java
├── repository/
│   ├── UserRepository.java
│   ├── PlantRepository.java
│   ├── CareTaskRepository.java
│   ├── HealthRecordRepository.java
│   ├── EnvironmentDataRepository.java
│   └── SpeciesRepository.java
├── entity/
│   ├── User.java
│   ├── Plant.java
│   ├── Species.java
│   ├── CareTask.java
│   ├── HealthRecord.java
│   ├── EnvironmentData.java
│   ├── Consultation.java
│   └── AuditLog.java
├── dto/
│   ├── request/
│   └── response/
├── exception/
│   └── GlobalExceptionHandler.java
```

```
| | PlantNotFoundException.java
| | CareScheduleException.java
| util/
| | JwtUtil.java
| | PlantCareUtil.java
| | DateUtil.java
```

4.1.2 Database Schema Design

Core Tables:

Users Table:

```
sql

CREATE TABLE users (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('ADMIN', 'PLANT_CARE_SPECIALIST', 'PREMIUM_PLANT_OWNER', 'STANDARD_PLANT_OWNER', 'COMMUNITY_MEMBER') NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  email_verified BOOLEAN DEFAULT FALSE,
  location VARCHAR(200),
  gardening_experience ENUM('BEGINNER', 'INTERMEDIATE', 'ADVANCED', 'EXPERT'),
  timezone VARCHAR(50),
  notification_preferences JSON,
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  last_login TIMESTAMP
);
```

Species Table:

```
sql
```

```
CREATE TABLE species (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  common_name VARCHAR(200) NOT NULL,  
  scientific_name VARCHAR(200) UNIQUE NOT NULL,  
  family_name VARCHAR(100),  
  care_difficulty ENUM('EASY', 'MODERATE', 'DIFFICULT', 'EXPERT'),  
  light_requirements ENUM('LOW', 'MEDIUM', 'HIGH', 'DIRECT'),  
  water_frequency_days INT,  
  humidity_min INT,  
  humidity_max INT,  
  temperature_min_celsius DECIMAL(5,2),  
  temperature_max_celsius DECIMAL(5,2),  
  soil_ph_min DECIMAL(3,1),  
  soil_ph_max DECIMAL(3,1),  
  growth_rate ENUM('SLOW', 'MODERATE', 'FAST'),  
  max_height_cm INT,  
  fertilizer_frequency_days INT,  
  pruning_frequency_days INT,  
  repotting_frequency_months INT,  
  common_issues JSON,  
  care_tips TEXT,  
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Plants Table:

```
sql
```

```

CREATE TABLE plants (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  owner_id BIGINT NOT NULL,
  species_id BIGINT NOT NULL,
  nickname VARCHAR(100) NOT NULL,
  location VARCHAR(200),
  acquisition_date DATE,
  current_height_cm DECIMAL(5,1),
  current_width_cm DECIMAL(5,1),
  pot_size VARCHAR(50),
  soil_type VARCHAR(100),
  last_watered_date TIMESTAMP,
  last_fertilized_date TIMESTAMP,
  last_pruned_date TIMESTAMP,
  last_repotted_date TIMESTAMP,
  health_status ENUM('EXCELLENT', 'GOOD', 'FAIR', 'POOR', 'CRITICAL'),
  notes TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (owner_id) REFERENCES users(id),
  FOREIGN KEY (species_id) REFERENCES species(id)
);

```

Care_Tasks Table:

```

sql

CREATE TABLE care_tasks (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  plant_id BIGINT NOT NULL,
  task_type ENUM('WATERING', 'FERTILIZING', 'PRUNING', 'REPOTTING', 'PEST_INSPECTION', 'HEALTH_CHECK', 'CLEAN'),
  scheduled_date TIMESTAMP NOT NULL,
  completed_date TIMESTAMP,
  status ENUM('PENDING', 'COMPLETED', 'SKIPPED', 'OVERDUE'),
  priority ENUM('LOW', 'MEDIUM', 'HIGH', 'URGENT'),
  notes TEXT,
  reminder_sent BOOLEAN DEFAULT FALSE,
  auto_generated BOOLEAN DEFAULT FALSE,
  completed_by BIGINT,
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (plant_id) REFERENCES plants(id),
  FOREIGN KEY (completed_by) REFERENCES users(id)
);

```

Environment_Data Table:

```
sql

CREATE TABLE environment_data (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  plant_id BIGINT,
  location_id VARCHAR(100),
  sensor_id VARCHAR(100),
  temperature_celsius DECIMAL(4,1),
  humidity_percentage DECIMAL(4,1),
  light_level_lux INT,
  soil_moisture_percentage DECIMAL(4,1),
  ph_level DECIMAL(3,1),
  recorded_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  data_source ENUM('MANUAL', 'SENSOR', 'WEATHER_API'),
  FOREIGN KEY (plant_id) REFERENCES plants(id),
  INDEX idx_plant_date (plant_id, recorded_date),
  INDEX idx_location_date (location_id, recorded_date)
);
```

Health_Records Table:

```
sql

CREATE TABLE health_records (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  plant_id BIGINT NOT NULL,
  assessment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  overall_health ENUM('EXCELLENT', 'GOOD', 'FAIR', 'POOR', 'CRITICAL'),
  symptoms JSON,
  diagnosed_issues JSON,
  treatments_applied JSON,
  specialist_id BIGINT,
  photos JSON,
  growth_measurements JSON,
  notes TEXT,
  follow_up_date DATE,
  recovery_status ENUM('NOT_APPLICABLE', 'RECOVERING', 'RECOVERED', 'WORSENING'),
  FOREIGN KEY (plant_id) REFERENCES plants(id),
  FOREIGN KEY (specialist_id) REFERENCES users(id)
);
```

4.1.3 JPQL Query Implementation

Custom Repository Methods:

@Repository

public interface PlantRepository extends JpaRepository<Plant, Long> {

@Query("SELECT p FROM Plant p WHERE p.owner.id = :ownerId " +
"AND p.isActive = true ORDER BY p.nickname ASC")

Page<Plant> findActiveByOwnerId(@Param("ownerId") Long ownerId, Pageable pageable);

@Query("SELECT p FROM Plant p JOIN p.species s WHERE p.owner.id = :ownerId " +
"AND (:nickname IS NULL OR LOWER(p.nickname) LIKE LOWER(CONCAT('%', :nickname, '%'))) " +
"AND (:speciesName IS NULL OR LOWER(s.commonName) LIKE LOWER(CONCAT('%', :speciesName, '%'))) " +
"AND (:healthStatus IS NULL OR p.healthStatus = :healthStatus) " +
"AND (:location IS NULL OR LOWER(p.location) LIKE LOWER(CONCAT('%', :location, '%'))) " +
"AND p.isActive = true")

Page<Plant> findPlantsWithFilters(

@Param("ownerId") Long ownerId,
@Param("nickname") String nickname,
@Param("speciesName") String speciesName,
@Param("healthStatus") HealthStatus healthStatus,
@Param("location") String location,
Pageable pageable

);

@Query("SELECT p FROM Plant p WHERE p.owner.id = :ownerId " +
"AND p.healthStatus IN ('POOR', 'CRITICAL') " +
"ORDER BY CASE p.healthStatus WHEN 'CRITICAL' THEN 1 WHEN 'POOR' THEN 2 END")

List<Plant> findPlantsNeedingAttention(@Param("ownerId") Long ownerId);

@Query("SELECT new com.plantcare.dto.PlantCareStatsDTO(" +
"COUNT(p), p.healthStatus, AVG(DATEDIFF(CURRENT_DATE, p.lastWateredDate))) " +
"FROM Plant p WHERE p.owner.id = :ownerId AND p.isActive = true " +
"GROUP BY p.healthStatus")

List<PlantCareStatsDTO> getPlantStatsByOwner(@Param("ownerId") Long ownerId);

}

@Repository

public interface CareTaskRepository extends JpaRepository<CareTask, Long> {

@Query("SELECT ct FROM CareTask ct JOIN ct.plant p " +
"WHERE p.owner.id = :ownerId " +
"AND ct.scheduledDate BETWEEN :startDate AND :endDate " +
"AND ct.status IN :statuses " +
"ORDER BY ct.scheduledDate ASC, ct.priority DESC")

List<CareTask> findUpcomingTasksByOwner(
@Param("ownerId") Long ownerId,
@Param("startDate") LocalDateTime startDate,
@Param("endDate") LocalDateTime endDate,

```

    @Param("statuses") List<TaskStatus> statuses
};

@Query("SELECT ct FROM CareTask ct JOIN ct.plant p " +
    "WHERE p.owner.id = :ownerid " +
    "AND ct.scheduledDate < :currentDate " +
    "AND ct.status = 'PENDING' " +
    "ORDER BY ct.priority DESC, ct.scheduledDate ASC")
List<CareTask> findOverdueTasksByOwner(
    @Param("ownerid") Long ownerid,
    @Param("currentDate") LocalDateTime currentDate
);

@Query("SELECT new com.plantcare.dto.TaskCompletionRateDTO(" +
    "ct.taskType, COUNT(ct), " +
    "SUM(CASE WHEN ct.status = 'COMPLETED' THEN 1 ELSE 0 END)) " +
    "FROM CareTask ct JOIN ct.plant p WHERE p.owner.id = :ownerid " +
    "AND ct.scheduledDate >= :fromDate " +
    "GROUP BY ct.taskType")
List<TaskCompletionRateDTO> getTaskCompletionRatesByType(
    @Param("ownerid") Long ownerid,
    @Param("fromDate") LocalDateTime fromDate
);
}

@Repository
public interface EnvironmentDataRepository extends JpaRepository<EnvironmentData, Long> {

    @Query("SELECT ed FROM EnvironmentData ed " +
        "WHERE ed.plant.id = :plantId " +
        "AND ed.recordedDate >= :fromDate " +
        "ORDER BY ed.recordedDate DESC")
    List<EnvironmentData> findRecentDataByPlant(
        @Param("plantId") Long plantId,
        @Param("fromDate") LocalDateTime fromDate
    );

    @Query("SELECT new com.plantcare.dto.EnvironmentTrendDTO(" +
        "DATE(ed.recordedDate), AVG(ed.temperatureCelsius), " +
        "AVG(ed.humidityPercentage), AVG(ed.soilMoisturePercentage)) " +
        "FROM EnvironmentData ed WHERE ed.plant.id = :plantId " +
        "AND ed.recordedDate >= :fromDate " +
        "GROUP BY DATE(ed.recordedDate) " +
        "ORDER BY DATE(ed.recordedDate)")
    List<EnvironmentTrendDTO> getEnvironmentTrends(
        @Param("plantId") Long plantId,
        @Param("fromDate") LocalDateTime fromDate
    );
}

```

```

    );
}

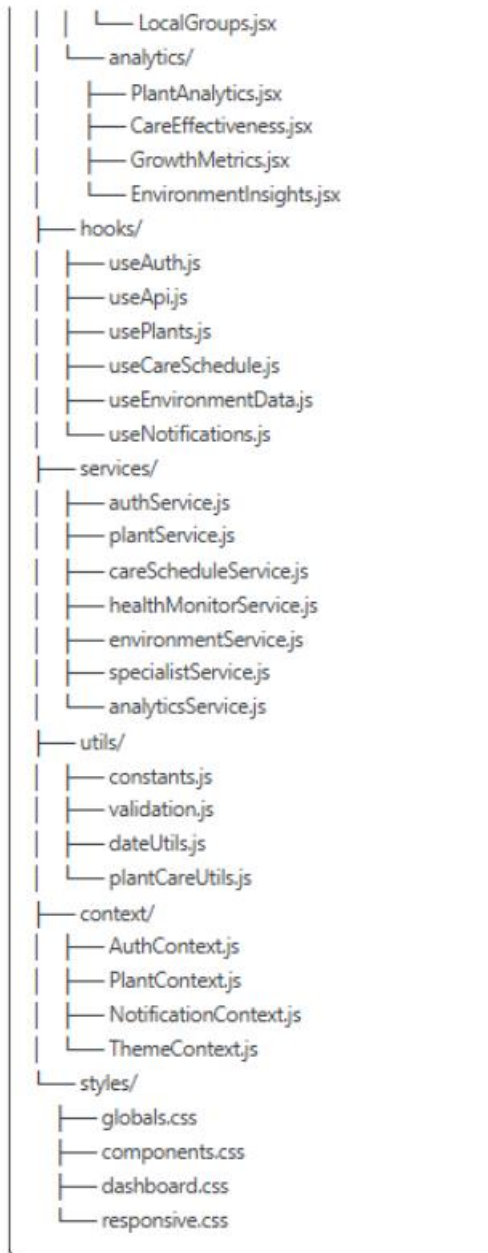
```

4.2 Frontend Architecture

4.2.1 React Application Structure

src/

```
├── components/
│   ├── common/
│   │   ├── Header.jsx
│   │   ├── Sidebar.jsx
│   │   ├── Footer.jsx
│   │   ├── LoadingSpinner.jsx
│   │   └── ErrorBoundary.jsx
│   ├── auth/
│   │   ├── LoginForm.jsx
│   │   ├── RegisterForm.jsx
│   │   └── ProtectedRoute.jsx
│   ├── dashboard/
│   │   ├── PlantOwnerDashboard.jsx
│   │   ├── SpecialistDashboard.jsx
│   │   └── AdminDashboard.jsx
│   ├── plants/
│   │   ├── PlantList.jsx
│   │   ├── PlantProfile.jsx
│   │   ├── PlantForm.jsx
│   │   ├── PlantGallery.jsx
│   │   └── PlantSearch.jsx
│   ├── care/
│   │   ├── CareScheduler.jsx
│   │   ├── TaskList.jsx
│   │   ├── CareCalendar.jsx
│   │   └── CareHistory.jsx
│   ├── health/
│   │   ├── HealthMonitor.jsx
│   │   ├── SymptomChecker.jsx
│   │   ├── TreatmentTracker.jsx
│   │   └── GrowthTracker.jsx
│   ├── environment/
│   │   ├── EnvironmentDashboard.jsx
│   │   ├── SensorData.jsx
│   │   ├── ClimateControl.jsx
│   │   └── EnvironmentHistory.jsx
│   ├── specialist/
│   │   ├── ConsultationScheduler.jsx
│   │   ├── ExpertAdvice.jsx
│   │   ├── CarePlanBuilder.jsx
│   │   └── ClientManager.jsx
│   ├── community/
│   │   ├── CommunityForum.jsx
│   │   ├── PlantSharing.jsx
│   │   └── KnowledgeBase.jsx
```



5. Security Requirements

5.1 Authentication and Authorization

5.1.1 JWT Implementation

java

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationEntryPoint jwtAuthenticationEntryPoint;

    @Autowired
    private JwtRequestFilter jwtRequestFilter;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/species/public").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/specialist/**").hasAnyRole("ADMIN", "PLANT_CARE_SPECIALIST")
                .requestMatchers("/api/premium/**").hasAnyRole("ADMIN", "PLANT_CARE_SPECIALIST", "PREMIUM_PLANT_Owner")
                .requestMatchers("/api/plants/**").hasAnyRole("ADMIN", "PLANT_CARE_SPECIALIST",
                    "PREMIUM_PLANT_OWNER", "STANDARD_PLANT_OWNER")
                .requestMatchers("/api/community/**").hasAnyRole("ADMIN", "PLANT_CARE_SPECIALIST",
                    "PREMIUM_PLANT_OWNER", "STANDARD_PLANT_OWNER", "COMMUNITY_MEMBER")
                .anyRequest().authenticated()
            )
            .exceptionHandling(ex -> ex.authenticationEntryPoint(jwtAuthenticationEntryPoint))
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

5.2 Role-Based Permission Matrix

Functionality	Guest	Community Member	Standard Plant Owner	Premium Plant Owner	Plant Care Specialist	Admin
Authentication & Profile						
View Public Plant Info	✓	✓	✓	✓	✓	✓
Register Account	✓	-	-	-	-	-
Login/Logout	-	✓	✓	✓	✓	✓
Update Profile	-	✓	✓	✓	✓	✓
Plant Management						
Create Plant Profiles	-	-	✓	✓	✓	✓
View Own Plants	-	-	✓	✓	✓	✓
View Client Plants	-	-	-	-	✓	✓
Advanced Plant Analytics	-	-	-	✓	✓	✓
Plant Collection Sharing	-	Limited	✓	✓	✓	✓
Care Scheduling						
Basic Care Reminders	-	-	✓	✓	✓	✓
Advanced Scheduling	-	-	-	✓	✓	✓
AI-Powered Recommendations	-	-	-	✓	✓	✓
Bulk Task Management	-	-	-	✓	✓	✓
Health Monitoring						
Basic Health Tracking	-	-	✓	✓	✓	✓
Disease Identification	-	-	Limited	✓	✓	✓
Treatment Planning	-	-	-	✓	✓	✓
Health Analytics	-	-	Basic	✓	✓	✓
Environmental Control						
Manual Data Entry	-	-	✓	✓	✓	✓
IoT Sensor Integration	-	-	-	✓	✓	✓
Automated Alerts	-	-	-	✓	✓	✓
Climate Optimization	-	-	-	✓	✓	✓
Specialist Services						
Browse Specialists	-	✓	✓	✓	-	✓
Book Consultations	-	-	-	✓	-	✓
Provide Consultations	-	-	-	-	✓	✓

Functionality	Guest	Community Member	Standard Plant Owner	Premium Plant Owner	Plant Care Specialist	Admin
Develop Care Plans	-	-	-	-	✓	✓
Community Features						
View Forums	✓	✓	✓	✓	✓	✓
Post in Forums	-	✓	✓	✓	✓	✓
Plant Trading	-	Limited	✓	✓	✓	✓
Expert Q&A Access	-	Limited	✓	✓	✓	✓
Analytics & Reporting						
Basic Plant Stats	-	-	✓	✓	✓	✓
Advanced Analytics	-	-	-	✓	✓	✓
Predictive Insights	-	-	-	✓	✓	✓
System Analytics	-	-	-	-	-	✓
Administrative Functions						
User Management	-	-	-	-	-	✓
System Configuration	-	-	-	-	-	✓
Content Moderation	-	-	-	-	Limited	✓
Audit Logs Access	-	-	-	-	-	✓

6. Performance Requirements

6.1 System Performance Specifications

- **Authentication:** Login/logout operations < 1 second for 95% of requests
- **Plant Management:** Plant creation and updates < 2 seconds for 90% of requests
- **Search Operations:** Plant and species search results < 1.5 seconds for complex queries
- **Care Scheduling:** Task generation and scheduling < 2 seconds for bulk operations
- **Health Monitoring:** Health assessment and photo analysis < 3 seconds
- **Environmental Data:** Sensor data processing < 500ms for real-time updates
- **Concurrent Users:** Support 2000+ concurrent users without performance degradation
- **Database Performance:** Handle 500,000+ plants and 50,000+ users with efficient querying
- **Mobile Performance:** Mobile application load time < 3 seconds on 4G networks
- **Real-time Updates:** Environmental alerts and notifications < 1 second latency

6.2 Scalability Requirements

- **Horizontal Scaling:** Support load balancing across multiple server instances
 - **Database Scaling:** Read replica support for analytics and environmental data
 - **IoT Data Processing:** Handle high-frequency sensor data with stream processing
 - **File Storage:** Scalable photo and document storage with CDN integration
 - **Cache Management:** Redis-based caching for frequently accessed plant data
-

7. UI/UX Requirements

7.1 Responsive Design Requirements

- **Mobile-First Design:** Optimized for mobile plant care activities
- **Breakpoint Support:** Desktop (1200px+), Tablet (768px-1199px), Mobile (320px-767px)
- **Touch-Friendly Interface:** Minimum 44px touch targets for mobile interactions
- **Cross-Browser Compatibility:** Support for Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Progressive Web App:** Offline capability for essential plant care functions

7.2 User Experience Guidelines

- **Intuitive Navigation:** Plant-centric information hierarchy and logical care flow
- **Visual Plant Recognition:** Photo-based plant identification and comparison
- **Care Workflow:** Guided care processes with step-by-step instructions
- **Accessibility:** WCAG 2.1 AA compliance for inclusive plant care management
- **Dark/Light Mode:** User preference-based theme switching with plant-friendly colors

7.3 Session Management

- **Context-Based Sessions:** Role-specific session durations and plant care continuity
 - **No Local Storage:** Sensitive plant and user data stored server-side with secure token management
 - **Session Timeout:** Automatic logout after inactivity (configurable by role and activity)
 - **Multi-Device Support:** Synchronized sessions across devices with care data consistency
-

8. Appendices

8.1 API Endpoint Specifications

8.1.1 Authentication Endpoints

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/refresh-token
POST /api/auth/forgot-password
POST /api/auth/reset-password
GET /api/auth/verify-email/{token}
```

8.1.2 Plant Management Endpoints

```
GET /api/plants?page={page}&size={size}&sort={sort}
POST /api/plants
GET /api/plants/{id}
PUT /api/plants/{id}
DELETE /api/plants/{id}
GET /api/plants/search?q={query}&filters={filters}
POST /api/plants/{id}/photos
GET /api/plants/{id}/photos
PUT /api/plants/{id}/health-status
GET /api/plants/{id}/care-history
POST /api/plants/{id}/notes
GET /api/plants/dashboard-summary
GET /api/plants/needing-attention
POST /api/plants/bulk-update
```

8.1.3 Species Database Endpoints

```
GET /api/species?page={page}&size={size}&difficulty={difficulty}
GET /api/species/{id}
GET /api/species/search?q={query}
GET /api/species/{id}/care-requirements
GET /api/species/{id}/common-issues
GET /api/species/popular
POST /api/species/{id}/care-tips
GET /api/species/categories
```

8.1.4 Care Scheduling Endpoints

```
GET  /api/care-tasks?plantId={id}&date={date}&status={status}
POST /api/care-tasks
GET  /api/care-tasks/{id}
PUT  /api/care-tasks/{id}
DELETE /api/care-tasks/{id}
PUT  /api/care-tasks/{id}/complete
GET  /api/care-tasks/upcoming
GET  /api/care-tasks/overdue
POST /api/care-tasks/bulk-complete
GET  /api/care-tasks/calendar/{month}/{year}
POST /api/care-tasks/generate-schedule
PUT  /api/care-tasks/{id}/reschedule
GET  /api/care-tasks/statistics
```

8.1.5 Health Monitoring Endpoints

```
GET  /api/health-records?plantId={id}&fromDate={date}
POST /api/health-records
GET  /api/health-records/{id}
PUT  /api/health-records/{id}
POST /api/health-records/{id}/photos
GET  /api/health-records/{id}/treatments
POST /api/health-records/{id}/treatments
PUT  /api/health-records/{id}/recovery-status
GET  /api/health-records/symptoms-checker
POST /api/health-records/diagnose
GET  /api/health-records/trends/{plantId}
```

8.1.6 Environmental Monitoring Endpoints

```
GET  /api/environment?plantId={id}&locationId={location}&fromDate={date}
POST /api/environment/manual-entry
GET  /api/environment/{plantId}/current
GET  /api/environment/{plantId}/trends
POST /api/environment/sensor-data
GET  /api/environment/alerts
PUT  /api/environment/alerts/{id}/acknowledge
GET  /api/environment/optimal-conditions/{speciesId}
POST /api/environment/calibrate-sensor
GET  /api/environment/sensors/status
```

8.1.7 Specialist Services Endpoints

```
GET  /api/specialists?specialty={type}&location={area}&rating={min}
GET  /api/specialists/{id}/profile
POST /api/consultations/book
GET  /api/consultations/{id}
PUT  /api/consultations/{id}/reschedule
POST /api/consultations/{id}/notes
GET  /api/consultations/my-appointments
PUT  /api/consultations/{id}/complete
POST /api/care-plans
GET  /api/care-plans/{id}
PUT  /api/care-plans/{id}
GET  /api/specialists/{id}/availability
POST /api/specialists/reviews
```

8.1.8 Community Endpoints

```
GET  /api/community/forums?category={type}&page={page}
POST /api/community/forums/{id}/posts
GET  /api/community/posts/{id}
PUT  /api/community/posts/{id}
DELETE /api/community/posts/{id}
POST /api/community/posts/{id}/reply
GET  /api/community/marketplace
POST /api/community/marketplace/listings
GET  /api/community/events?location={area}
POST /api/community/groups/join/{id}
GET  /api/community/knowledge-base
POST /api/community/knowledge-base/contribute
```

8.1.9 Analytics Endpoints

```
GET  /api/analytics/plants/{ownerId}/summary
GET  /api/analytics/plants/{id}/growth-trends
GET  /api/analytics/care-effectiveness/{ownerId}
GET  /api/analytics/environment/{plantId}/optimization
GET  /api/analytics/specialist/{id}/performance
GET  /api/analytics/system/usage
POST /api/reports/generate
GET  /api/reports/{id}/download
GET  /api/analytics/predictions/{plantId}
```

8.1.10 Administrative Endpoints

```
GET /api/admin/users?page={page}&role={role}&status={status}
POST /api/admin/users
PUT /api/admin/users/{id}
DELETE /api/admin/users/{id}
GET /api/admin/analytics/system
GET /api/admin/audit-logs
POST /api/admin/notifications/broadcast
GET /api/admin/reports/usage
PUT /api/admin/system/settings
GET /api/admin/health-check
POST /api/admin/species/approve
GET /api/admin/content/moderation
```

8.2 Database Schema Implementation

8.2.1 Additional Supporting Tables

Consultations Table:

```
sql
CREATE TABLE consultations (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  client_id BIGINT NOT NULL,
  specialist_id BIGINT NOT NULL,
  plant_id BIGINT,
  consultation_type ENUM('GENERAL', 'HEALTH_ISSUE', 'CARE_PLAN', 'EMERGENCY'),
  scheduled_date TIMESTAMP NOT NULL,
  duration_minutes INT DEFAULT 60,
  status ENUM('SCHEDULED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED', 'NO_SHOW'),
  meeting_link VARCHAR(500),
  consultation_notes TEXT,
  care_recommendations TEXT,
  follow_up_required BOOLEAN DEFAULT FALSE,
  follow_up_date DATE,
  rating INT CHECK (rating BETWEEN 1 AND 5),
  client_feedback TEXT,
  fee_amount DECIMAL(8,2),
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (client_id) REFERENCES users(id),
  FOREIGN KEY (specialist_id) REFERENCES users(id),
  FOREIGN KEY (plant_id) REFERENCES plants(id)
);
```

Care_Plans Table:

sql

```
CREATE TABLE care_plans (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  plant_id BIGINT NOT NULL,  
  specialist_id BIGINT,  
  plan_name VARCHAR(200) NOT NULL,  
  plan_description TEXT,  
  start_date DATE NOT NULL,  
  end_date DATE,  
  watering_schedule JSON,  
  fertilizing_schedule JSON,  
  pruning_schedule JSON,  
  monitoring_requirements JSON,  
  environmental_targets JSON,  
  special_instructions TEXT,  
  is_active BOOLEAN DEFAULT TRUE,  
  success_metrics JSON,  
  progress_notes TEXT,  
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (plant_id) REFERENCES plants(id),  
  FOREIGN KEY (specialist_id) REFERENCES users(id)  
);
```

Community_Posts Table:

sql

```
CREATE TABLE community_posts (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  author_id BIGINT NOT NULL,  
  forum_category ENUM('GENERAL', 'PLANT_ID', 'CARE_TIPS', 'PROBLEMS', 'SHOWCASE', 'TRADING'),  
  title VARCHAR(300) NOT NULL,  
  content TEXT NOT NULL,  
  plant_id BIGINT,  
  tags JSON,  
  images JSON,  
  upvotes INT DEFAULT 0,  
  downvotes INT DEFAULT 0,  
  view_count INT DEFAULT 0,  
  is_pinned BOOLEAN DEFAULT FALSE,  
  is_locked BOOLEAN DEFAULT FALSE,  
  status ENUM('PUBLISHED', 'DRAFT', 'MODERATED', 'REMOVED'),  
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (author_id) REFERENCES users(id),  
  FOREIGN KEY (plant_id) REFERENCES plants(id)  
);
```

Notifications Table:

sql

```

CREATE TABLE notifications (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  type ENUM('CARE_REMINDER', 'HEALTH_ALERT', 'ENVIRONMENT_WARNING', 'CONSULTATION_REMINDER',
    'COMMUNITY_UPDATE', 'SYSTEM_ANNOUNCEMENT', 'PLANT_MILESTONE'),
  title VARCHAR(200) NOT NULL,
  message TEXT NOT NULL,
  priority ENUM('LOW', 'MEDIUM', 'HIGH', 'URGENT'),
  plant_id BIGINT,
  related_entity_type VARCHAR(50),
  related_entity_id BIGINT,
  action_url VARCHAR(500),
  is_read BOOLEAN DEFAULT FALSE,
  scheduled_date TIMESTAMP,
  sent_date TIMESTAMP,
  expires_date TIMESTAMP,
  delivery_methods JSON,
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (plant_id) REFERENCES plants(id)
);

```

Plant_Photos Table:

```

sql

CREATE TABLE plant_photos (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  plant_id BIGINT NOT NULL,
  health_record_id BIGINT,
  file_name VARCHAR(255) NOT NULL,
  file_path VARCHAR(500) NOT NULL,
  file_size BIGINT NOT NULL,
  mime_type VARCHAR(100) NOT NULL,
  photo_type ENUM('GENERAL', 'HEALTH_ISSUE', 'GROWTH_PROGRESS', 'BEFORE_TREATMENT', 'AFTER_TREATMENT'),
  description TEXT,
  taken_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  camera_settings JSON,
  tags JSON,
  is_public BOOLEAN DEFAULT FALSE,
  created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (plant_id) REFERENCES plants(id),
  FOREIGN KEY (health_record_id) REFERENCES health_records(id)
);

```

8.3 Advanced JPQL Implementations

8.3.1 Complex Analytics Queries

```
java

// Care effectiveness analysis
@Query("SELECT ct.taskType, " +
    "COUNT(ct) as totalTasks, " +
    "SUM(CASE WHEN ct.status = 'COMPLETED' THEN 1 ELSE 0 END) as completedTasks, " +
    "AVG(CASE WHEN ct.completedDate IS NOT NULL " +
    " THEN TIMESTAMPDIFF(HOUR, ct.scheduledDate, ct.completedDate) " +
    " ELSE NULL END) as avgCompletionDelayHours " +
    "FROM CareTask ct JOIN ct.plant p WHERE p.ownerId = :ownerId " +
    "AND ct.scheduledDate >= :fromDate " +
    "GROUP BY ct.taskType " +
    "ORDER BY completedTasks DESC")
List<Object[]> getCareEffectivenessAnalysis(@Param("ownerId") Long ownerId,
    @Param("fromDate") LocalDateTime fromDate);

// Plant health correlation with care frequency
@Query("SELECT p.id, p.nickname, p.healthStatus, " +
    "COUNT(ct) as totalCareTasks, " +
    "AVG(ed.soilMoisturePercentage) as avgSoilMoisture, " +
    "AVG(ed.temperatureCelsius) as avgTemperature " +
    "FROM Plant p " +
    "LEFT JOIN CareTask ct ON p.id = ct.plant.id AND ct.status = 'COMPLETED' " +
    "LEFT JOIN EnvironmentData ed ON p.id = ed.plant.id AND ed.recordedDate >= :fromDate " +
    "WHERE p.ownerId = :ownerId " +
    "GROUP BY p.id, p.nickname, p.healthStatus " +
    "ORDER BY p.healthStatus DESC, totalCareTasks DESC")
List<Object[]> getPlantHealthCareCorrelation(@Param("ownerId") Long ownerId,
    @Param("fromDate") LocalDateTime fromDate);

// Environmental optimization recommendations
@Query("SELECT s.commonName, s.lightRequirements, s.temperatureMinCelsius, s.temperatureMaxCelsius, " +
    "AVG(ed.temperatureCelsius) as currentAvgTemp, " +
    "AVG(ed.humidityPercentage) as currentAvgHumidity, " +
    "AVG(ed.lightLevelLux) as currentAvgLight " +
    "FROM Plant p JOIN p.species s " +
    "LEFT JOIN EnvironmentData ed ON p.id = ed.plant.id AND ed.recordedDate >= :fromDate " +
    "WHERE p.ownerId = :ownerId " +
    "GROUP BY s.id, s.commonName, s.lightRequirements, s.temperatureMinCelsius, s.temperatureMaxCelsius " +
    "HAVING (currentAvgTemp < s.temperatureMinCelsius OR currentAvgTemp > s.temperatureMaxCelsius)")
List<Object[]> getEnvironmentOptimizationOpportunities(@Param("ownerId") Long ownerId,
    @Param("fromDate") LocalDateTime fromDate);
```

8.3.2 Custom Search and Filter Implementations

```
java
// Advanced plant search with multiple criteria
@Query("SELECT DISTINCT p FROM Plant p " +
    "JOIN p.species s " +
    "LEFT JOIN p.healthRecords hr " +
    "LEFT JOIN p.careTasks ct " +
    "WHERE p.ownerId = :ownerId " +
    "AND (:nickname IS NULL OR LOWER(p.nickname) LIKE LOWER(CONCAT('%', :nickname, '%')))" +
    "AND (:speciesName IS NULL OR LOWER(s.commonName) LIKE LOWER(CONCAT('%', :speciesName, '%')))" +
    "AND (:location IS NULL OR LOWER(p.location) LIKE LOWER(CONCAT('%', :location, '%')))" +
    "AND (:healthStatus IS NULL OR p.healthStatus = :healthStatus)" +
    "AND (:careDifficulty IS NULL OR s.careDifficulty = :careDifficulty)" +
    "AND (:hasIssues IS NULL OR " +
    "    (:hasIssues = true AND hr.overallHealth IN ('POOR', 'CRITICAL')) OR " +
    "    (:hasIssues = false AND (hr.id IS NULL OR hr.overallHealth NOT IN ('POOR', 'CRITICAL'))))" +
    "AND (:needsCare IS NULL OR " +
    "    (:needsCare = true AND ct.status = 'PENDING' AND ct.scheduledDate <= CURRENT_TIMESTAMP) OR " +
    "    (:needsCare = false)))")
Page<Plant> searchPlantsAdvanced(
    @Param("ownerId") Long ownerId,
    @Param("nickname") String nickname,
    @Param("speciesName") String speciesName,
    @Param("location") String location,
    @Param("healthStatus") HealthStatus healthStatus,
    @Param("careDifficulty") CareDifficulty careDifficulty,
    @Param("hasIssues") Boolean hasIssues,
    @Param("needsCare") Boolean needsCare,
    Pageable pageable
);
```

8.4 Frontend Implementation Guidelines

8.4.1 Role-Based Routing Implementation

```
jsx
```

```

// App.jsx - Main routing with role-based access
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { useAuth } from './hooks/useAuth';
import ProtectedRoute from './components/auth/ProtectedRoute';

function App() {
  const { user, loading } = useAuth();

  if (loading) return <LoadingSpinner />;

  return (
    <Router>
      <div className="app">
        <Routes>
          { /* Public Routes */ }
          <Route path="/" element={<LandingPage />} />
          <Route path="/login" element={<LoginForm />} />
          <Route path="/register" element={<RegisterForm />} />
          <Route path="/species/public" element={<PublicSpeciesDatabase />} />

          { /* Protected Routes */ }
          <Route path="/dashboard" element={
            <ProtectedRoute allowedRoles={['STANDARD_PLANT_OWNER', 'PREMIUM_PLANT_OWNER', 'PLANT_CARE_SPE
              <DashboardRouter />
            </ProtectedRoute>
          } />

          <Route path="/plants/*" element={
            <ProtectedRoute allowedRoles={['STANDARD_PLANT_OWNER', 'PREMIUM_PLANT_OWNER', 'PLANT_CARE_SPE
              <PlantRouter />
            </ProtectedRoute>
          } />

          <Route path="/specialist/*" element={
            <ProtectedRoute allowedRoles={['PLANT_CARE_SPECIALIST', 'ADMIN']}>
              <SpecialistRouter />
            </ProtectedRoute>
          } />

          <Route path="/premium/*" element={
            <ProtectedRoute allowedRoles={['PREMIUM_PLANT_OWNER', 'PLANT_CARE_SPECIALIST', 'ADMIN']}>
              <PremiumRouter />
            </ProtectedRoute>
          } />

          <Route path="/admin/*" element={

```

```
    <ProtectedRoute allowedRoles={['ADMIN']}>
      <AdminRouter />
    </ProtectedRoute>
  } />

  <Route path="/community/*" element={
    <ProtectedRoute allowedRoles={['COMMUNITY_MEMBER', 'STANDARD_PLANT_OWNER', 'PREMIUM_PLANT_O
      <CommunityRouter />
    </ProtectedRoute>
  } />
</Routes>
</div>
</Router>
);
}
```

8.4.2 Context-Based Session Management

jsx

```

// AuthContext.js - Secure session management with plant care focus
import { createContext, useContext, useReducer, useEffect } from 'react';
import { authService } from '../services/authService';

const AuthContext = createContext();

const authReducer = (state, action) => {
  switch (action.type) {
    case 'LOGIN_SUCCESS':
      return {
        ...state,
        user: action.payload.user,
        token: action.payload.token,
        isAuthenticated: true,
        loading: false,
        plantPreferences: action.payload.user.plantPreferences
      };
    case 'LOGOUT':
      return {
        user: null,
        token: null,
        isAuthenticated: false,
        loading: false,
        plantPreferences: null
      };
    case 'UPDATE_PREFERENCES':
      return {
        ...state,
        plantPreferences: action.payload.preferences
      };
    case 'SESSION_EXPIRED':
      return {
        user: null,
        token: null,
        isAuthenticated: false,
        loading: false,
        sessionExpired: true
      };
    default:
      return state;
  }
};

export const AuthProvider = ({ children }) => {
  const [state, dispatch] = useReducer(authReducer, {
    user: null,

```

```

    token: null,
    isAuthenticated: false,
    loading: true,
    sessionExpired: false,
    plantPreferences: null
  });

  // Role-specific session timeouts
  const getSessionTimeout = () => {
    switch (state.user?.role) {
      case 'ADMIN':
        return 4 * 60 * 60 * 1000; // 4 hours
      case 'PLANT_CARE_SPECIALIST':
        return 8 * 60 * 60 * 1000; // 8 hours
      case 'PREMIUM_PLANT_OWNER':
        return 12 * 60 * 60 * 1000; // 12 hours
      case 'STANDARD_PLANT_OWNER':
        return 24 * 60 * 60 * 1000; // 24 hours
      default:
        return 2 * 60 * 60 * 1000; // 2 hours
    }
  };

  // Auto-refresh token before expiration
  useEffect(() => {
    if (state.token && state.user) {
      const timeout = getSessionTimeout();
      const refreshInterval = Math.min(timeout / 4, 15 * 60 * 1000); // Refresh every 15 min or 1/4 session time

      const intervalId = setInterval(() => {
        authService.refreshToken()
          .then(response => {
            dispatch({ type: 'TOKEN_REFRESH', payload: response.data });
          })
          .catch(() => {
            dispatch({ type: 'SESSION_EXPIRED' });
          });
      }, refreshInterval);

      return () => clearInterval(intervalId);
    }
  }, [state.token, state.user]);

  const login = async (credentials) => {
    try {
      const response = await authService.login(credentials);
      dispatch({ type: 'LOGIN_SUCCESS', payload: response.data });
    }
  }

```

```

        return response.data;
    } catch (error) {
        throw error;
    }
};

const logout = async () => {
    try {
        await authService.logout();
    } catch (error) {
        console.error('Logout error:', error);
    } finally {
        dispatch({ type: 'LOGOUT' });
    }
};

const updatePlantPreferences = (preferences) => {
    dispatch({ type: 'UPDATE_PREFERENCES', payload: { preferences } });
};

return (
    <AuthContext.Provider value={{
        ...state,
        login,
        logout,
        updatePlantPreferences
    }}>
        {children}
    </AuthContext.Provider>
);
};

export const useAuth = () => {
    const context = useContext(AuthContext);
    if (!context) {
        throw new Error('useAuth must be used within an AuthProvider');
    }
    return context;
};

```

8.5 Validation and Exception Handling

8.5.1 Custom Validation Annotations

```
java
```

```

// Custom validation for plant care data
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = PlantCareScheduleValidator.class)
public @interface ValidPlantCareSchedule {
    String message() default "Invalid care schedule configuration";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

@Component
public class PlantCareScheduleValidator implements ConstraintValidator<ValidPlantCareSchedule, CareTask> {

    @Override
    public boolean isValid(CareTask careTask, ConstraintValidatorContext context) {
        if (careTask.getScheduledDate() == null) {
            return false;
        }

        // Validate that care task is not scheduled in the past (except for overdue tracking)
        if (careTask.getScheduledDate().isBefore(LocalDateTime.now().minusHours(1))) {
            return false;
        }

        // Validate task type compatibility with plant species
        if (careTask.getPlant() != null && careTask.getPlant().getSpecies() != null) {
            return isTaskTypeValidForSpecies(careTask.getTaskType(), careTask.getPlant().getSpecies());
        }

        return true;
    }

    private boolean isTaskTypeValidForSpecies(TaskType taskType, Species species) {
        // Implementation for species-specific task validation
        switch (taskType) {
            case WATERING:
                return species.getWaterFrequencyDays() != null && species.getWaterFrequencyDays() > 0;
            case FERTILIZING:
                return species.getFertilizerFrequencyDays() != null && species.getFertilizerFrequencyDays() > 0;
            case PRUNING:
                return species.getPruningFrequencyDays() != null && species.getPruningFrequencyDays() > 0;
            default:
                return true;
        }
    }
}

```

8.5.2 Global Exception Handling

```
java
```

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(PlantNotFoundException.class)
    public ResponseEntity<ErrorResponse> handlePlantNotFound(PlantNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(
            "PLANT_NOT_FOUND",
            ex.getMessage(),
            System.currentTimeMillis()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(CareScheduleException.class)
    public ResponseEntity<ErrorResponse> handleCareScheduleError(CareScheduleException ex) {
        ErrorResponse error = new ErrorResponse(
            "CARE_SCHEDULE_ERROR",
            ex.getMessage(),
            System.currentTimeMillis()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }

    @ExceptionHandler(EnvironmentDataException.class)
    public ResponseEntity<ErrorResponse> handleEnvironmentDataError(EnvironmentDataException ex) {
        ErrorResponse error = new ErrorResponse(
            "ENVIRONMENT_DATA_ERROR",
            ex.getMessage(),
            System.currentTimeMillis()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }

    @ExceptionHandler(SpecialistNotAvailableException.class)
    public ResponseEntity<ErrorResponse> handleSpecialistNotAvailable(SpecialistNotAvailableException ex) {
        ErrorResponse error = new ErrorResponse(
            "SPECIALIST_NOT_AVAILABLE",
            ex.getMessage(),
            System.currentTimeMillis()
        );
        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ValidationErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        ValidationErrorResponse error = new ValidationErrorResponse(

```

```
        "VALIDATION_FAILED",
        "Plant care data validation failed"
    );

    ex.getBindingResult().getFieldErrors().forEach(fieldError -> {
        error.addFieldError(fieldError.getField(), fieldError.getDefaultMessage());
    });

    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
}
}
```

8.6 Deployment and Configuration

8.6.1 Application Properties

```
properties
```

```
# application.yml
spring:
  application:
    name: plant-care-scheduler

  datasource:
    url: ${DB_URL;jdbc:mysql://localhost:3306/plant_care}
    username: ${DB_USERNAME;plant_care_user}
    password: ${DB_PASSWORD;secure_password}
    driver-class-name: com.mysql.cj.jdbc.Driver

  jpa:
    hibernate:
      ddl-auto: ${DDL_AUTO;validate}
    show-sql: ${SHOW_SQL:false}
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQL8Dialect
        format_sql: true

  security:
    jwt:
      secret: ${JWT_SECRET;your-256-bit-secret-key}
      expiration: ${JWT_EXPIRATION;43200000} # 12 hours
      refresh-expiration: ${JWT_REFRESH_EXPIRATION;2592000000} # 30 days

  servlet:
    multipart:
      max-file-size: 20MB
      max-request-size: 100MB

  mail:
    host: ${MAIL_HOST;smtp.gmail.com}
    port: ${MAIL_PORT;587}
    username: ${MAIL_USERNAME;your-email@gmail.com}
    password: ${MAIL_PASSWORD;your-app-password}
    properties:
      mail:
        smtp:
          auth: true
          starttls:
            enable: true

  websocket:
    allowed-origins: ${WEBSOCKET_ORIGINS:http://localhost:3000}
```

```
logging:
  level:
    com.plantcare: ${LOG_LEVEL:INFO}
    org.springframework.security: ${SECURITY_LOG_LEVEL:WARN}
  pattern:
    file: "%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n"
  file:
    name: logs/plant-care.log

# Custom application properties
app:
  file-storage:
    upload-dir: ${UPLOAD_DIR:/uploads}
    max-file-size: 20MB

  notification:
    email:
      enabled: ${EMAIL_NOTIFICATIONS:true}
      from: ${EMAIL_FROM:noreply@plantcare.com}
    sms:
      enabled: ${SMS_NOTIFICATIONS:false}
      provider: ${SMS_PROVIDER:twilio}

  iot:
    enabled: ${IOT_ENABLED:false}
    mqtt-broker: ${MQTT_BROKER:tcp://localhost:1883}
    sensor-data-retention-days: ${SENSOR_RETENTION:90}

  analytics:
    retention-days: ${ANALYTICS_RETENTION:365}

  session:
    timeout:
      standard-user: ${STANDARD_SESSION_TIMEOUT:86400} # 24 hours
      premium-user: ${PREMIUM_SESSION_TIMEOUT:43200} # 12 hours
      specialist: ${SPECIALIST_SESSION_TIMEOUT:28800} # 8 hours
      admin: ${ADMIN_SESSION_TIMEOUT:14400} # 4 hours

  care-scheduling:
    auto-generate: ${AUTO_GENERATE_TASKS:true}
    advance-days: ${SCHEDULE_ADVANCE_DAYS:7}
    reminder-hours-before: ${REMINDER_HOURS:2}
```

8.7 Testing Strategy

8.7.1 Unit Test Examples

```

@ExtendWith(MockitoExtension.class)
class PlantServiceTest {

    @Mock
    private PlantRepository plantRepository;

    @Mock
    private SpeciesRepository speciesRepository;

    @Mock
    private CareScheduleService careScheduleService;

    @InjectMocks
    private PlantService plantService;

    @Test
    void createPlant_ValidInput_ReturnsPlant() {
        // Given
        Long ownerId = 1L;
        PlantCreateRequest request = new PlantCreateRequest();
        request.setSpeciesId(1L);
        request.setNickname("My Fiddle Leaf Fig");
        request.setLocation("Living Room");
        request.setAcquisitionDate(LocalDate.now().minusDays(30));

        Species species = new Species();
        species.setId(1L);
        species.setCommonName("Fiddle Leaf Fig");
        species.setCareDifficulty(CareDifficulty.MODERATE);

        Plant savedPlant = new Plant();
        savedPlant.setId(1L);
        savedPlant.setNickname("My Fiddle Leaf Fig");

        when(speciesRepository.findById(1L)).thenReturn(Optional.of(species));
        when(plantRepository.save(any(Plant.class))).thenReturn(savedPlant);

        // When
        PlantResponse response = plantService.createPlant(ownerId, request);

        // Then
        assertThat(response).isNotNull();
        assertThat(response.getId()).isEqualTo(1L);
        assertThat(response.getNickname()).isEqualTo("My Fiddle Leaf Fig");
        verify(careScheduleService).generateInitialCareSchedule(any(Plant.class));
    }
}

```

```

@Test
void getPlantsWithFilters_ValidFilters_ReturnsPagedResults() {
    // Given
    Long ownerId = 1L;
    PlantSearchRequest searchRequest = new PlantSearchRequest();
    searchRequest.setHealthStatus(HealthStatus.GOOD);
    Pageable pageable = PageRequest.of(0, 10);

    List<Plant> plants = Arrays.asList(new Plant(), new Plant());
    Page<Plant> page = new PageImpl<>(plants, pageable, plants.size());

    when(plantRepository.findPlantsWithFilters(
        eq(ownerId), any(), any(), eq(HealthStatus.GOOD), any(), eq(pageable)
    )).thenReturn(page);

    // When
    Page<PlantResponse> result = plantService.searchPlants(ownerId, searchRequest, pageable);

    // Then
    assertThat(result.getContent()).hasSize(2);
    assertThat(result.getTotalElements()).isEqualTo(2);
}
}

```

8.7.2 Integration Test Examples

```

java

```

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@TestPropertySource(locations = "classpath:application-test.properties")
@Transactional
class PlantControllerIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Autowired
    private PlantRepository plantRepository;

    @Autowired
    private UserRepository userRepository;

    @Test
    void createPlant_AuthenticatedUser_ReturnsCreatedPlant() {
        // Given
        String token = authenticateAndGetToken("plantowner@test.com", "password");

        PlantCreateRequest request = new PlantCreateRequest();
        request.setSpeciesId(1L);
        request.setNickname("Test Plant");
        request.setLocation("Test Location");
        request.setAcquisitionDate(LocalDate.now());

        HttpHeaders headers = new HttpHeaders();
        headers.setBearerAuth(token);
        HttpEntity<PlantCreateRequest> entity = new HttpEntity<>(request, headers);

        // When
        ResponseEntity<PlantResponse> response = restTemplate.exchange(
            "/api/plants",
            HttpMethod.POST,
            entity,
            PlantResponse.class
        );

        // Then
        assertThat(response.getStatusCode(), isEqualTo(HttpStatus.CREATED));
        assertThat(response.getBody(), isNotNull());
        assertThat(response.getBody().getNickname(), isEqualTo("Test Plant"));
    }

    private String authenticateAndGetToken(String email, String password) {
        LoginRequest loginRequest = new LoginRequest(email, password);
        ResponseEntity<AuthResponse> authResponse = restTemplate.postForEntity(

```

```
        "/api/auth/login", loginRequest, AuthResponse.class
    );
    return authResponse.getBody().getToken();
}
}
```

8.8 IoT Integration

8.8.1 Sensor Data Processing

```
java
```

```

@Component
@Slf4j
public class SensorDataProcessor {

    @Autowired
    private EnvironmentDataRepository environmentDataRepository;

    @Autowired
    private NotificationService notificationService;

    @EventListener
    @Async
    public void processSensorData(SensorDataEvent event) {
        try {
            SensorData data = event.getSensorData();

            // Validate sensor data
            if (!isValidSensorData(data)) {
                log.warn("Invalid sensor data received: {}", data);
                return;
            }

            // Save to database
            EnvironmentData environmentData = mapToEnvironmentData(data);
            environmentDataRepository.save(environmentData);

            // Check for alerts
            checkForEnvironmentAlerts(environmentData);

            // Update plant health indicators
            updatePlantHealthIndicators(environmentData);

        } catch (Exception e) {
            log.error("Error processing sensor data", e);
        }
    }

    private void checkForEnvironmentAlerts(EnvironmentData data) {
        if (data.getPlant() != null) {
            Species species = data.getPlant().getSpecies();

            // Temperature alerts
            if (data.getTemperatureCelsius() != null) {
                if (data.getTemperatureCelsius() < species.getTemperatureMinCelsius() ||
                    data.getTemperatureCelsius() > species.getTemperatureMaxCelsius()) {

```

```
        notificationService.sendEnvironmentAlert(  
            data.getPlant().getOwner().getId(),  
            "Temperature out of optimal range for " + data.getPlant().getNickname(),  
            NotificationPriority.HIGH  
        );  
    }  
}  
  
// Soil moisture alerts  
if (data.getSoilMoisturePercentage() != null && data.getSoilMoisturePercentage() < 20) {  
    notificationService.sendEnvironmentAlert(  
        data.getPlant().getOwner().getId(),  
        data.getPlant().getNickname() + " needs watering - soil moisture is low",  
        NotificationPriority.MEDIUM  
    );  
}  
}  
}
```

8.9 Business Rules and Validation

8.9.1 Complex Business Logic

```
java
```

```

@Component
public class PlantCareBusinessRules {

    public void validatePlantCreation(PlantCreateRequest request, User owner) {
        // Rule: Standard users limited to 20 plants, Premium users to 100 plants
        long currentPlantCount = getCurrentPlantCount(owner.getId());

        if (owner.getRole() == UserRole.STANDARD_PLANT_OWNER && currentPlantCount >= 20) {
            throw new BusinessException(
                "Standard users are limited to 20 plants. Upgrade to Premium for more plants.");
        }

        if (owner.getRole() == UserRole.PREMIUM_PLANT_OWNER && currentPlantCount >= 100) {
            throw new BusinessException(
                "Premium users are limited to 100 plants.");
        }

        // Rule: Plant nickname must be unique within user's collection
        if (isNicknameAlreadyUsed(owner.getId(), request.getNickname())) {
            throw new BusinessException(
                "You already have a plant with this nickname. Please choose a different name.");
        }
    }

    public void validateCareTaskScheduling(CareTask task) {
        // Rule: Cannot schedule watering more frequently than species minimum interval
        if (task.getTaskType() == TaskType.WATERING) {
            LocalDateTime lastWatering = getLastWateringDate(task.getPlant().getId());
            int speciesMinInterval = task.getPlant().getSpecies().getWaterFrequencyDays();

            if (lastWatering != null &&
                task.getScheduledDate().isBefore(lastWatering.plusDays(speciesMinInterval - 1))) {
                throw new BusinessException(
                    "Watering scheduled too frequently for this plant species. " +
                    "Minimum interval: " + speciesMinInterval + " days");
            }
        }

        // Rule: Cannot schedule care tasks more than 30 days in advance
        if (task.getScheduledDate().isAfter(LocalDate.now().plusDays(30))) {
            throw new BusinessException(
                "Care tasks cannot be scheduled more than 30 days in advance");
        }
    }

    public void validateSpecialistConsultation(ConsultationRequest request, User client) {

```

```

// Rule: Standard users limited to 1 consultation per month
if (client.getRole() == UserRole.STANDARD_PLANT_OWNER) {
    long monthlyConsultations = getMonthlyConsultationCount(client.getId());
    if (monthlyConsultations >= 1) {
        throw new BusinessException(
            "Standard users are limited to 1 consultation per month. " +
            "Upgrade to Premium for unlimited consultations.");
    }
}

// Rule: Emergency consultations must be scheduled within 24 hours
if (request.getConsultationType() == ConsultationType.EMERGENCY) {
    if (request.getScheduledDate().isAfter(LocalDateTime.now().plusHours(24))) {
        throw new BusinessException(
            "Emergency consultations must be scheduled within 24 hours");
    }
}
}
}

```

8.10 Future Enhancement Roadmap

8.10.1 Planned Features by Release

Phase 2.0 - Advanced Analytics and AI (Q1 2026)

- Machine learning-based plant health prediction
- AI-powered disease identification from photos
- Automated care optimization based on environmental data
- Predictive growth modeling and harvest timing
- Advanced analytics dashboard with ML insights

Phase 2.1 - Mobile Application (Q2 2026)

- Native iOS and Android applications
- Offline plant care tracking with sync capabilities
- Camera-based plant identification and health assessment
- Push notifications for care reminders and alerts
- Location-based gardening recommendations

Phase 3.0 - IoT and Automation (Q3 2026)

- Smart irrigation system integration
- Automated lighting control based on plant needs

- Advanced sensor network with wireless connectivity
- Climate control system integration
- Robotic care assistant integration

Phase 3.1 - Community and Marketplace (Q4 2026)

- Plant trading and marketplace platform
- Local gardening community features
- Expert-led virtual workshops and courses
- Plant care certification programs
- Gamification with achievement systems

Document Version Control

Version	Date	Changes	Author
1.0	August 13, 2025	Initial comprehensive SRS document for Plant Care Scheduler	Development Team

End of Document

This Software Requirements Specification provides comprehensive guidelines for developing a complete Plant Care Scheduler application. All requirements, architectural patterns, and implementation details are designed to enable full application development based solely on this documentation.